

An automatic solver for very large jigsaw puzzles using genetic algorithms

Dror Sholomon¹ · Omid E. David¹ · Nathan S. Netanyahu^{1,2}

Received: 8 August 2015 / Revised: 3 December 2015
© Springer Science+Business Media New York 2016

Abstract In this paper we propose the first effective genetic algorithm (GA)-based jigsaw puzzle solver. We introduce a novel crossover procedure that merges two “parent” solutions to an improved “child” configuration by detecting, extracting, and combining correctly assembled puzzle segments. The solver proposed exhibits state-of-the-art performance, as far as handling previously attempted puzzles more accurately and efficiently, as well puzzle sizes that have not been attempted before. The extended experimental results provided in this paper include, among others, a thorough inspection of up to 30,745-piece puzzles (compared to previous attempts on 22,755-piece puzzles), using a considerably faster concurrent implementation of the algorithm. Furthermore, we explore the impact of different phases of the novel crossover operator by experimenting with several variants of the GA. Finally, we compare different fitness functions and their effect on the overall results of the GA-based solver.

Keywords Computer vision · Genetic algorithms · Jigsaw puzzle

A preliminary version of this paper appeared in *Proceedings of the IEEE Computer Vision and Pattern Recognition Conference* [19].

Area Editor for Games: Moshe Sipper.

✉ Omid E. David
mail@omiddavid.com

Dror Sholomon
dror.sholomon@gmail.com

Nathan S. Netanyahu
nathan@cs.biu.ac.il; nathan@cfar.umd.edu

¹ Department of Computer Science, Bar-Ilan University, 52900 Ramat Gan, Israel

² Center for Automation Research, University of Maryland, College Park, MD 20742, USA

1 Introduction

The problem domain of jigsaw puzzles is widely known to almost every human being from childhood. Given n different non-overlapping pieces of an image, the player has to reconstruct the original image, taking advantage of both the shape and chromatic information of each piece. Although this popular game was proven to be an NP-complete problem [1, 7], it has been played successfully by children worldwide. Solutions to this problem might benefit the fields of biology [13], chemistry [24], literature [15], speech descrambling [26], archeology [2, 12], image editing [5] and the recovery of shredded documents or photographs [3, 6, 11, 14]. Besides, as Goldberg et al. [10] have noted, the jigsaw puzzle problem may and should be researched for the sole reason that it stirs pure interest.

Recent years have witnessed a vast improvement in the research and development of automatic jigsaw puzzle solvers, manifested in both puzzle size, solution accuracy, and amount of manual human intervention required. Reported state-of-the-art solvers are fully automated and can handle puzzles of up to 9000 pieces. Most, if not all, of the aforementioned solvers are greedy, and thus are at great risk of converging to local optima. Despite the great potential in devising a genetic algorithm (GA)-based solver, the success of previous attempts was limited to 64-piece puzzles [23], most likely due to the enormous complexity associated with

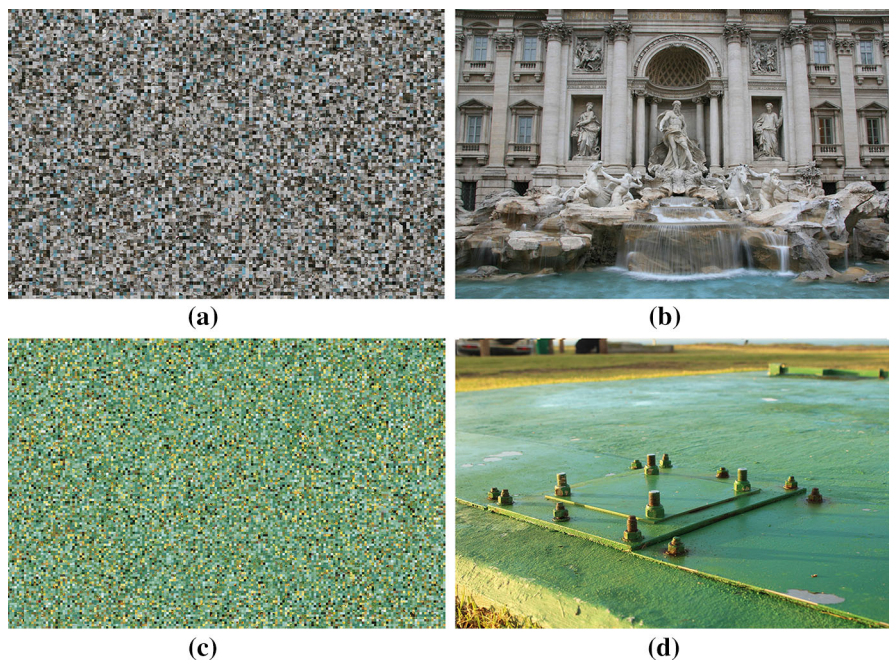


Fig. 1 Jigsaw puzzles before and after reassembly using our GA-based solver: **a, b** 10,375-piece puzzles and **c, d** 22,755-piece puzzles, which are among the largest automatically solved puzzles to date

evolutionary computation (EC), in general, and the inherent difficulty of devising an effective crossover operator for this problem, in particular.

Following the convention used in recent work [4, 16, 25], we consider jigsaw puzzles with non-overlapping, (28×28) square pieces, where both piece orientation and puzzle dimensions are known. This version, where piece locations are the only unknowns, is called a “Type 1” puzzle [9]. The solver has no knowledge of the original image and may not make any use of it. In this paper we introduce a novel crossover operator, enabling for the first time an effective GA-based puzzle solver. Our solver compromises neither speed nor size as it outperforms, for the most part, state-of-the-art solvers, tackling successfully up to 30,745-piece size puzzles (i.e. more than three times the number of pieces that has ever been attempted/reported), within a reasonable time frame (see e.g. Fig. 1 for 10,375- and 22,755-piece puzzles). Our contribution should benefit research regarding EC in general, and the jigsaw puzzle problem, in particular. From an EC perspective, our novel techniques could be used for solving additional problems with similar properties. As to the jigsaw puzzle problem, our proposed framework could prove useful for solving more advanced variants, such as puzzles with missing pieces, unknown piece orientation, mixed puzzles, and more. Finally, we assemble a new benchmark, consisting of sets of larger images (with varying degrees of difficulty), which we make public to the community [18]. Also, we share all of our results (on this benchmark and other public datasets) for future testing and comparative evaluation of jigsaw puzzle solvers.

This paper is an extended version of our previously presented work [19]. We lay out the requirements from an effective (GA-based) jigsaw puzzle solver, and provide a more detailed description of our crossover operator, as part of our proposed solution. Furthermore, the paper includes new empirical results of an extended set of experiments aimed at a more exhaustive performance evaluation of our presented solver, in general, and its novel crossover operator, in particular. We formed an extended benchmark of up to 30,745-piece puzzles and tested our solver’s performance multiple times on each image. Specifically, we have pursued the following empirical issues: (1) Explored the relative impact of the different phases of our 3-phase crossover operator; (2) tested our solver’s performance also with a different fitness function; (3) investigated further the shifting anomaly discovered in our early work; (4) introduced a concurrent crossover version to reduce significantly the overall run-time of the GA (without affecting the solver’s accuracy).

2 Previous work

Jigsaw puzzles were first introduced around 1760 by John Spilsbury, a Londonian engraver and mapmaker. Nevertheless, the first attempt by the scientific community to computationally solve the problem is attributed to Freeman and Garder [8] who in 1964 presented a solver which could handle up to nine-piece problems. Ever since then, the research focus regarding the problem has shifted from shape-based to merely color-based solvers of square-tile puzzles. In 2010 Cho et al. [4] presented a

probabilistic puzzle solver that could handle up to 432 pieces, given some a priori knowledge of the puzzle. Their results were improved a year later by Yang et al. [25] who presented a particle filter-based solver. Furthermore, Pomeranz et al. [16] introduced that year, for the first time, a fully automated square jigsaw puzzle solver that could handle puzzles of up to 3000 pieces. Gallagher [9] has further advanced this by considering a more general variant of the problem, where neither piece orientation nor puzzle dimensions are known.

In its most basic form, every puzzle solver requires an estimation function to evaluate the compatibility of adjacent pieces and a strategy for placing the pieces as accurately as possible. Although much effort has been invested in perfecting the compatibility functions, recent strategies tend to be greedy, which is known to be problematic when encountering local optima. Thus, despite achieving very good—if not perfect—solutions for some puzzles, supplementary material provided by Pomeranz et al. [17] suggests there is much room for improvement for many other puzzles. Comparative studies conducted by Gallagher ([9]; Table 4), regarding the benchmark of 432-piece images, reveal only a slight improvement in accuracy relatively to Pomeranz et al. (95.1 vs. 95.0 %). To the best of our knowledge, no additional runs on other benchmarks have been reported by Gallagher. Interestingly, despite the availability of puzzle solvers for 3000- and 9000-piece puzzles, there exists no image set, for the purpose of benchmark testing, containing puzzles with more than 805 pieces. Current state-of-the-art solvers were tested only on very few large images, and those tested contained an extreme variety of textures and colors, which renders them, admittedly, as “easier” for solving [9]. We assume that as with the smaller images, the accuracy of current solvers on some large puzzles could be greatly improved by using more sophisticated algorithms.

New attempts were made to solve the jigsaw puzzle problem since our original publication [19]; see e.g. [22] which is based on the so-called loop constraints. To the best of our knowledge, our GA-based solver exhibits comparable or superior performance (in most cases) to other “Type 1” solvers reported in the literature. Furthermore, the proposed GA framework has been adapted successfully to solve considerably harder puzzle variants, e.g. where the pieces are taken from different puzzles and where the piece orientations and puzzle dimensions are not known. See [20, 21] for further details.

3 GA-based puzzle solver

We use a standard implementation of GA with fitness proportional solution (also known as roulette wheel selection) and uniform crossover. In particular, we use a measure of elitism as in each generation we replicate the best 4 chromosomes. Mutation is embedded inside the crossover operator (which thus might be also referred to as a variation operator). Algorithm 1 describes the above mentioned framework. The following parameter values are used:

population size = 1000
 number of generations = 100
 mutation rate = 0.05

Algorithm 1 Pseudocode of GA Framework

```

1: population  $\leftarrow$  generate 1000 random chromosomes
2: for generation_number = 1  $\rightarrow$  100 do
3:   evaluate all chromosomes using the fitness function
4:   new_population  $\leftarrow$  NULL
5:   copy 4 best chromosomes to new_population
6:   while size(new_population)  $\leq$  1000 do
7:     parent1  $\leftarrow$  select chromosome
8:     parent2  $\leftarrow$  select chromosome
9:     child  $\leftarrow$  crossover(parent1, parent2)
10:    add child to new_population
11:   end while
12:   population  $\leftarrow$  new_population
13: end for

```

Given a puzzle (image) of $(N \times M)$ pieces, we represent a chromosome by an $(N \times M)$ matrix, each entry of which corresponds to a piece number (a piece is assigned a number according to its initial location in the given puzzle). Thus, each chromosome represents a complete solution to the jigsaw puzzle problem, i.e. a suggested placement of all pieces.

Having provided a framework overview, we now describe in greater detail the various critical components of the GA proposed, e.g. the chromosome representation, fitness function, and crossover operator.

3.1 The fitness function

Each chromosome represents a suggested placement of all pieces. The problem variant at hand assumes no knowledge whatsoever of the original image and thus, the correctness of the absolute location of puzzle pieces cannot be estimated in a simple manner. Instead, the pairwise compatibility (defined below) of every pair of adjacent pieces is computed.

A measure which ranks the likelihood of two pieces in the original image as adjacent is called *compatibility*. Let $C(x_i, x_j, R)$ denote the compatibility of two puzzle pieces x_i, x_j , where $R \in \{l, r, a, b\}$ indicates the spatial relation between these pieces, i.e. whether x_j lies, respectively, to the left/right of x_i , or above or below it.

Cho et al. [4] explored five possible compatibility measures, of which the *dissimilarity measure* of Eq. (1) was shown to be the most discriminative. Pomeranz et al. [16] further investigated this issue and chose a similar dissimilarity measure with some slight optimizations. The dissimilarity measure relies on the premise that adjacent jigsaw pieces in the original image tend to share similar colors along their abutting edges, i.e. the sum (over all neighboring pixels) of squared color differences (over all color bands) should be minimal. Assuming pieces x_i, x_j are represented in normalized L*a*b* space by a $K \times K \times 3$ matrix, where K is the height/width of a piece (in pixels), their dissimilarity (if x_j is to the right of x_i , for example) is

$$D(x_i, x_j, r) = \sqrt{\sum_{k=1}^K \sum_{cb=1}^3 (x_i(k, K, cb) - x_j(k, 1, cb))^2}, \quad (1)$$

where cb stands for color band. It is important to note that dissimilarity is not a metric, since almost always $D(x_i, x_j, R) \neq D(x_j, x_i, R)$. Obviously, to maximize the compatibility of two pieces, their dissimilarity should be minimized.

Another important consideration in choosing a fitness function is the run-time cost. Since every chromosome in every generation must be evaluated, a fitness function must be relatively computationally-inexpensive. We chose the standard dissimilarity, as it meets this criterion and also seems to be sufficiently discriminative. To speed up further the computation of the fitness function we added a lookup table of size $2 \cdot (N \cdot M)^2$ containing all of the pairwise compatibilities for all pieces (we need keep only compatibilities with respect to “right” and “above”, as those with respect to “left” and “below” can be easily deduced).

Finally, the fitness function of a given chromosome is the sum of pairwise dissimilarities over all neighboring pieces (whose configuration is represented by the chromosome). Representing a chromosome by an $(N \times M)$ matrix, where a matrix entry $x_{i,j}$ ($i = 1 \dots N, j = 1 \dots M$) corresponds to a single puzzle piece, we define its fitness as

$$\sum_{i=1}^N \sum_{j=1}^{M-1} (D(x_{i,j}, x_{i,j+1}, r)) + \sum_{i=1}^{N-1} \sum_{j=1}^M (D(x_{i,j}, x_{i+1,j}, b)), \quad (2)$$

where, as previously indicated, r and b stand for “right” and “below”, respectively.

3.2 The crossover operator

3.2.1 Problem definition

As noted above, a chromosome is represented by an $(N \times M)$ matrix, where each matrix entry corresponds to the number of a puzzle tile. This representation is straightforward and lends itself easily to the evaluation of the fitness function described above. The main issue concerning this representation is the design of an appropriate crossover operator. A naive crossover operator with respect to the given representation will create a new child chromosome at random, such that each entry of the resulting matrix is the corresponding cell of the first or second parent. This approach yields usually a child chromosome with duplicate and/or missing puzzle pieces, which is of course an invalid solution to the problem. The inherent difficulty surrounding the crossover operator may well have played a critical role in delaying thus far the development of a state-of-the-art solution to the problem, based on evolutionary computation.

Once the validity issue is rectified, one needs to consider very carefully the crossover operator. Recall, crossover is applied to two chromosomes selected due to their high fitness values, where the fitness function used is an overall pairwise compatibility measure of adjacent puzzle pieces. At best, the function rewards a

correct placement of neighboring pieces next to each other, but it has no way of identifying the actual correct location of a piece. Since the population starts out from a random piece placement and then improves gradually, it is reasonable to assume that some correctly assembled puzzle segments emerge over the generations. Taking into account the fitness function's inability to reward a correct position, we expect such segments to appear most likely at incorrect locations. A crossover operator should pass on "good traits" from the parents to the child, thereby creating possibly a better solution. Discovering a correct segment is not trivial; it should be regarded as a good trait that needs to be exploited. Namely, it should not be discarded, but rather trickled to the next generations. Thus, the crossover operator must allow for *position independence*, i.e. the ability of shifting entire correctly-assembled segments, in an attempt to place them correctly in their true location in the child chromosome.

Once the position-independence issue is taken care of, one should address the issue of detecting correctly-assembled segments, which are possibly misplaced. What segment should the crossover operator pass on to an offspring? A random approach might seem appealing, but in practice it could be infeasible due to the enormous size of the solution domain of the problem. Some heuristics may be applied to distinguish correct segments from incorrect ones.

In summary, a good crossover operator should address the issues of validity of child chromosomes, detection of supposedly correctly-assembled puzzle segments in parents, and position independence of these segments when passed on to an offspring.

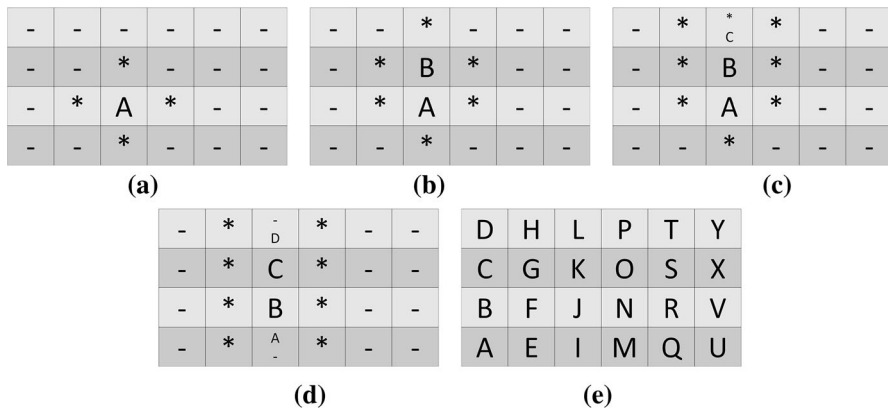


Fig. 2 Slots that may and may not be assigned a piece by crossover operator are marked, respectively, by "*" and "-" (the latter, since slots are not adjacent to any assigned piece or they are beyond image boundary): **a** Kernel's first stage, with only a single piece, **b–d** assignment of next three pieces; note shift downward of entire kernel in **(d)**, i.e. piece A does not end up where it is placed initially, and **e** resulting image, where entire column in **(d)** was shifted to the left

3.2.2 Our proposed solution

Given two parent chromosomes, i.e. two complete different arrangements of all puzzle pieces, the crossover operator constructs a child chromosome in a kernel-growing fashion, using both parents as “consultants”. The operator starts with a single piece and gradually joins other pieces at available kernel boundaries. New pieces may be joined only adjacently to existing ones, so that the emerging image is always contiguous. For example, let A be the first piece chosen by the operator. The second piece B must be placed in a neighboring slot, i.e. left/right of, above or below piece A . Assuming it is placed just above A , then a third piece C must be assigned to one of the empty, previously mentioned slots (i.e. left/right of, or below A), or to any of the newly available slots that are left/right of, or above B . See Fig. 2 for an illustration of the above scenario.

The operator keeps adding tiles from a bank of available puzzle pieces until no pieces are left. Hence, every piece will appear exactly once in the resulting image.

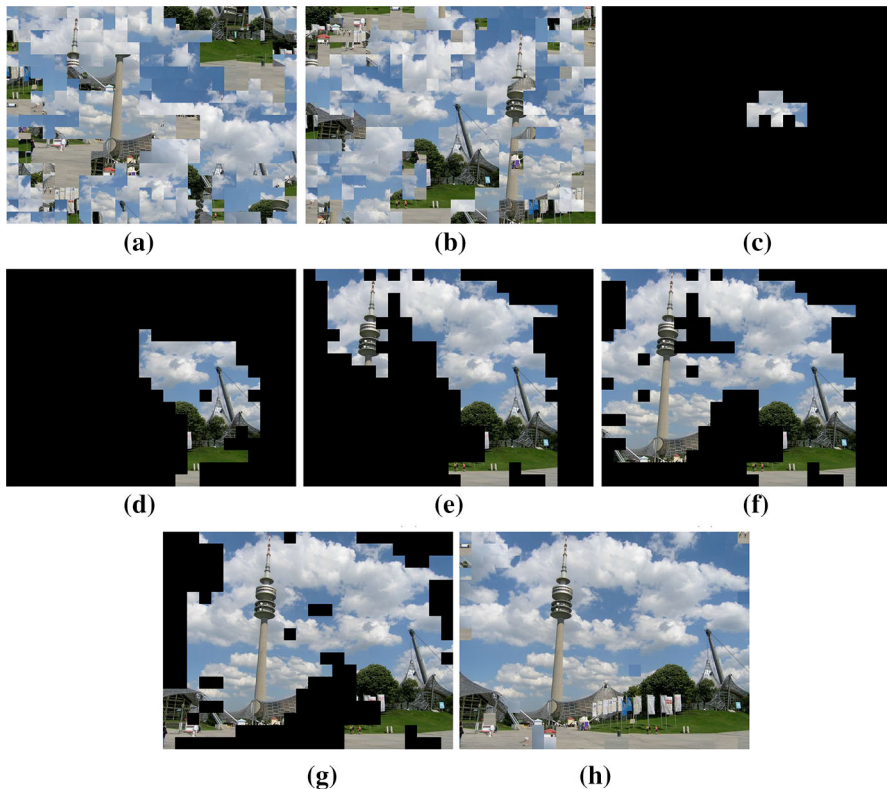


Fig. 3 Illustration of crossover operation: given **a** Parent1 and **b** Parent2, **c–g** depict gradual growth of kernel of pieces until **h** complete child obtained; note the detection of tower parts in both parents, which are shifted and merged into complete tower; image shifting during kernel growing is due to so-called piece-position independence

Since the image size is known in advance, the operator can ensure that there is no boundary violation. Thus, since the operator uses every piece exactly once inside a bounded frame of the correct size, achieving a valid image is guaranteed. Figure 3 illustrates the above described kernel-growing process.

A key property of the kernel-growing technique is that the location of every piece is determined only when the kernel reaches its final size and the child chromosome is complete. Before that, all pieces might be shifted, depending on the kernel's growth vector. The initial piece, for example, might end up at the lower-left corner of the image, if the kernel should grow only upwards and to the right, after this piece is first assigned. On the other hand, the very same piece might be placed ultimately at the center of the image, its upper-right corner, or any other location, for that matter. This change in a tile's location is illustrated in Figs. 2 and 3; see, in particular, Fig. 3(f)–(g) of the kernel-growing process, where pieces keep shifting to the right due to the insertion of new pieces on the left. It is this important property which enables the position independence of image segments. Namely, a correctly-assembled puzzle segment in a parent chromosome, possibly misplaced with respect to its true location, could be copied over during the crossover operation (while preserving its structure) to its correct location in an offspring.

Having described how to construct a child chromosome as a growing kernel, we now turn to the following remaining issues: (1) Which piece to select from the available bank of tiles and (2) where to place it in the child (i.e. at which available neighboring slot). Given a kernel, i.e. a partial image, we can mark all the boundaries where a new piece might be placed. A piece boundary is denoted by a pair (x_i, R) , consisting of the piece number and a spatial relation. The operator invokes a three-phase procedure. First, given all existing boundaries, the operator checks whether there exists a piece boundary which both parents agree on, i.e. whether there exists a piece x_j that is in the spatial direction R of x_i in both parents. If so, then x_j is added to the kernel in the appropriate location. If the parents agree on two or more boundaries, one of them is chosen at random and the corresponding piece is assigned. Obviously, if the piece is already in use, it cannot be (re)assigned, i.e. it is ignored as if the parents do not agree on that particular boundary. If there is no agreement between the parents on any piece at any boundary, the second phase begins. To understand this phase, we briefly review the concept of a *best-buddy* piece, first introduced by Pomeranz et al. [16]. Two pieces are said to be best buddies if each piece considers the other as its most compatible piece. Pieces x_i and x_j are said to be best buddies if

$$\begin{aligned} \forall x_k \in \text{Pieces}, C(x_i, x_j, R_1) \geq C(x_i, x_k, R_1) \\ \text{and} \\ \forall x_p \in \text{Pieces}, C(x_j, x_i, R_2) \geq C(x_j, x_p, R_2) \end{aligned} \quad (3)$$

where *Pieces* is the set of all given image pieces and R_1 and R_2 are “complementary” spatial relations (e.g. if $R_1 = \text{right}$, then $R_2 = \text{left}$ and vice versa). In the second phase, the operator checks whether one of the parents contains a piece x_j in a spatial relation R of x_i which is also a best buddy of x_i with respect to that relation. If so, the piece is chosen and assigned. As before, if multiple best-buddy pieces are

available, one of them is chosen at random. If a best-buddy piece is found that is already assigned, it is ignored and the search continues for other best-buddy pieces. If no best-buddy piece exists, the operator proceeds to the final third phase, where it selects a boundary at random and assigns to it the most compatible piece available. To introduce mutation, the operator places, with low probability, in the first and last phase, an available piece at random, instead of the most compatible relevant piece available.

In summary, the operator uses repeatedly a three-phase procedure of piece selection and assignment, placing first an agreed-upon piece, then a best-buddy piece, and finally the most compatible piece available (i.e. the most compatible piece not yet assigned). An assignment is only considered at relevant boundaries to maintain the contiguity of the kernel-growing image. The procedure returns to the first phase after every piece assignment due to the creation of new prospective boundaries. Algorithm 2 provides a simplified description of the crossover operator (without mutation).

Algorithm 2 Crossover operator simplified

- 1: If any available boundary meets the criterion of Phase 1 (both parents agree on a piece), place the piece there and goto (1); otherwise continue.
 - 2: If any available boundary meets the criterion of Phase 2 (one parent contains a best-buddy piece), place the piece there and goto (1); otherwise continue.
 - 3: Randomly choose a boundary, place the most compatible available piece there and goto (1).
-

See Fig. 4 for the algorithm's operation on several examples.

3.2.3 Rationale

We expect to see child chromosomes inherit “good” traits from their parent chromosomes in an effective GA framework. Since the algorithm encourages piece position independence, the trait of interest is captured by sets of neighboring pieces, rather than the locations of particular pieces. Correct puzzle segments correspond to the correct placement of pieces relatively to each other. The notion, for example, that piece x_i is in a spatial relation R to piece x_j is key to solving the jigsaw puzzle problem. Thus, although every chromosome accounts for a complete placement of all the pieces, it is the internal relative piece assignments that are examined and exploited. We assume that a trait common to both parents under consideration has propagated through the generations and is essentially the reason for their survival and selection. In other words, if both parents agree on a spatial relation, the algorithm would consider it true, and attempt to draw on it, with high probability. Given the highly randomized nature of the first few generations, agreed-upon pieces selected during this stage could persist incorrectly through the latter generations. Note, however, that due to the kernel-growing algorithm, some agreed-upon pieces might be selected as most compatible pieces at other boundaries and thus be discarded for later use. For example, let pieces A and B be adjacent in both parents;

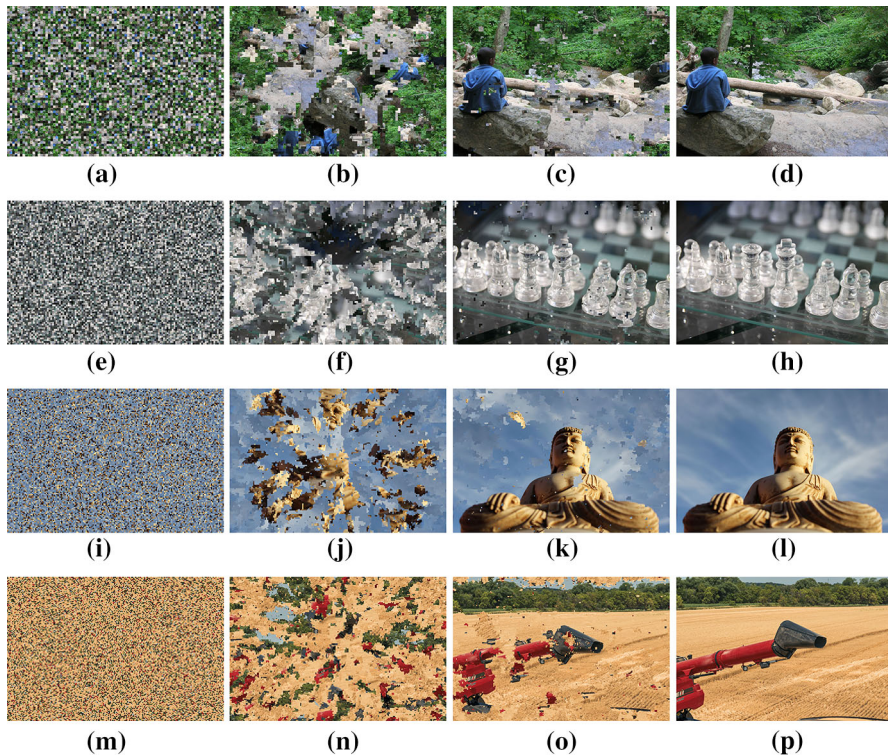


Fig. 4 Selected results of our GA-based solver for large puzzles. The first row shows: **a** 5015-piece puzzle and the best chromosome achieved by the GA in the **b** first, **c** second, and **d** last generation. Similarly, the second and third rows show the best chromosomes in the same generations for 10,375-, 22,755- and 30,745-piece puzzles, respectively. The accuracy of all puzzle solutions shown is 100 %. **a** 5015 pieces. **b** Generation 1. **c** Generation 2. **d** Final. **e** 10,375 pieces. **f** Generation 1. **g** Generation 2. **h** Final. **i** 22,755 pieces. **j** Generation 1. **k** Generation 2. **l** Final. **m** 30,745 pieces. **n** Generation 1. **o** Generation 2. **p** Final

piece *B* could be selected during the kernel growing and placed next to a different piece *C* in the kernel. In other words, agreed-upon relations formed at early generations might not necessarily carry over to the child.

If the parents agree on no piece, the algorithm could place a piece from one of the parents (picked randomly), that is compatible with a kernel piece at an available boundary. Another possibility would be to place a best-buddy piece (if one exists) with respect to a kernel piece at an available boundary. Since neither two adjacent pieces in a parent (which could have been assigned at random) nor two best-buddy pieces alone are guaranteed to capture a correct match, the idea in the second phase is to combine these two elements. In other words, the fact that two pieces are both adjacent in a parent and are also best buddies is a good indication that they probably match.

The rationale of the above two phases is based on the assumption that every chromosome contains some correct segments. Propagating such segments from

parents to their children is at the heart of the GA. More specifically, if two parents each contain a correct segment, with an overlap between the segments, the idea is to copy the joint sub-segment to the child, in the first phase, and add pieces from both parents, independently, in the second phase, to obtain the union of the segments. Combining the segments into a larger correct segment would yield an enhanced child chromosome, thereby advancing the overall correct solution.

The third phase ensures that the algorithm can always add a piece to the growing kernel (in case the conditions for phase 1 or phase 2 are not met), by placing the most compatible piece left with respect to an available boundary. In essence, this phase enables the GA to try many different greedy placements across each generation; placements that seem correct are likely to persist through the latter generations. This exemplifies the principle of propagation of good traits in the spirit of the theory of natural selection.

4 Experimental results

Cho et al. [4] introduced the following two main measures, which have been repeatedly used in previous works, to evaluate the accuracy of an assembled puzzle; the *direct comparison*, which measures the fraction of pieces located in their correct location, and the *neighbor comparison*, which measures the fraction of correct neighbors. The direct method is considered [16] less accurate and less meaningful due to its inadequate evaluation of slightly shifted puzzle solutions. Figure 5 illustrates the drawbacks of the direct comparison and the advantage of the neighbor comparison. Note that a 100 % score for a given piece configuration according to one definition implies also a perfect score according to the other criterion (and vice versa), i.e. the complete reconstruction of the original image. Thus, unless otherwise stated, all results are presented with respect to the neighbor comparison; for the sake of completeness, our results with respect to the direct comparison are reported in Table 1.

We first tested the proposed GA on the set of images supplied by Cho et al. [4] and all image sets supplied by Pomeranz et al. [16] (recall, all images are composed of 28×28 -pixel patches). The image data experimented with contain 20-image sets of 432-, 504-, and 805-piece puzzles and 3-image sets of 2360- and 3360-piece puzzles. Next, we have augmented the above benchmark by compiling four additional 20-image sets for this work and future studies. These additional sets contain images of 5015-, 10,375-, 22,755-, and 30,745-piece puzzles (note that the latter two puzzle sizes have not been attempted before). We ran the GA ten times on each image (see Fig. 4 for selected results), every time with a different random seed, and recorded—for these ten runs—the best, worst, and average accuracy (as well as the standard deviation). The averages of these best, worst, and average results are referred to as “average best”, “average worst”, and “average average”, respectively. Tables 1 and 2 list the results achieved by our GA on each set for the direct comparison and the neighbor comparison, respectively. Interestingly, despite the inherent randomness of GAs, different runs yield almost identical results, attesting to the solver’s robustness. Having observed little difference between the best and worst runs for the largest size puzzles, in particular, we settled for only five GA runs



Fig. 5 **a–d** Shifted solutions created by our GA; the accuracy for each solution is 0 % according to the direct comparison, but over 95 % according to the (more meaningful) neighbor comparison; amazingly, the dissimilarity of each solution is smaller than that of its original image counterpart

Table 1 Accuracy results using our GA: Averages of best, worst, and average scores (as well as standard deviations) are given for each image set under *direct comparison*; GA is invoked five times on each image of 22,755- and 30,745-piece puzzles and ten times on all other images

No. of Pieces	Avg. best (%)	Avg. worst (%)	Avg. avg. (%)	Avg. Standard Deviation (%)
432	86.19	80.56	82.94	2.62
540	92.75	90.57	91.65	0.65
805	94.67	92.79	93.63	0.62
2360	85.73	82.73	84.62	0.86
3300	89.92	65.42	86.62	7.19
5015	94.78	90.76	92.04	1.74
10,375	97.69	96.08	97.12	0.45
22,755	93.41	87.04	90.49	2.59
30,745	90.76	81.19	86.11	4.51

for each image of the 22,755- and 30,745-piece puzzles (note that the GA achieves very high accuracy also on these very large image sizes that have not been attempted before).

Table 2 Accuracy results using our GA: Averages of best, worst, and average scores (as well as standard deviations) are given for each image set under *neighbor comparison*; GA is invoked five times on each image of 22,755- and 30,745-piece puzzles and ten times on all other images

No. of Pieces	Avg. best (%)	Avg. worst (%)	Avg. avg. (%)	Avg. Standard Deviation (%)
432	96.16	95.21	95.70	0.34
540	95.96	94.65	95.38	0.40
805	96.26	95.35	95.85	0.31
2360	88.86	87.52	88.00	0.38
3300	92.76	91.91	92.37	0.27
5015	95.25	94.87	95.06	0.11
10,375	98.47	98.20	98.36	0.08
22,755	96.40	95.97	96.19	0.12
30,745	93.40	93.12	93.27	0.11

Table 3 Comparison of our accuracy results to those of Pomeranz et al. (derived from their supplementary material); averages are given for each image set of best scores over ten runs for each image

No. of Pieces	Pomeranz et al. (%)	GA (%)	Diff. (%)
432	94.25	96.16	1.91
540	90.90	95.96	5.06
805	89.70	96.26	6.56
2360	84.67	88.86	4.19
3300	85.00	92.76	7.76

Table 3 compares—for each image set—our average best results to those of Pomeranz et al. which can be easily derived from their well-documented results [17]. As can be seen, our GA results are more accurate than theirs. Nevertheless, as noted in the beginning of this paper, previous solvers might perform well on some images and worse on others. The advantageous performance of our solver is further demonstrated in Table 4, relating only to the three least accurately solved images in every set. Our solver gains a significant improvement of up to 21 % (for the 540-piece puzzle set); for some puzzles, the improvement was even 30 %. In Table 5 we provide a detailed comparison of our results, for 432- to 30K-piece puzzles, with those obtained by Pomeranz et al. [16], Gallagher [9], and Son et al. [22], based on the principle of so-called loop constraints [22]. The comprehensive

Table 4 Comparison of our accuracy results to those of Pomeranz et al. relating only to the three least accurately solved images in every set

No. of Pieces	Pomeranz et al. (%)	GA (%)	Diff. (%)
432	76.00	81.06	5.06
540	58.33	79.32	20.99
805	67.33	86.30	18.97
2360	84.67	88.86	4.19
3300	85.00	92.76	7.76

Table 5 Comparison of our accuracy results to those of Pomeranz et al. [16], Gallagher [9] and Son et al. [22]; Best results for each image set appear in bold

# of Pieces	Pomeranz et al. (%)	Gallagher (%)	Son et al. (%)	GA (%)
432	94.25	95.10	95.50	96.16
540	90.90	–	95.20	95.96
805	89.70	–	94.90	96.26
2360	84.67	–	96.40	88.86
3300	85.00	–	96.40	92.76
5015	–	–	–	95.25
10,375	–	–	–	98.47
22,755	–	–	–	96.40
30,745	–	–	–	93.40

set of results demonstrates that the proposed GA remains a state-of-the-art solver. Detailed results of the exact accuracy of every run on every image can be found in our supplementary material [18].

An interesting phenomenon observed is depicted in Fig. 5. For some puzzles, the GA obtains a “better-than-perfect” score, i.e. it returns a configuration for which the dissimilarity is smaller than that of the original (correct) image. Such tile (mis)placements were reproduced even with more sophisticated metrics such as the compatibility measure proposed in [16]. Moreover, in some cases, these abnormal solutions are arrived at after the GA does find the correct solution. As far as we know, observations of this kind have not been reported before. Although undesirable, this phenomenon attests to the GA’s capability of optimizing the overall cost function for a given image, thereby avoiding potentially local optima. Obviously, revisiting the fitness function (i.e. compatibility measure) would be required in an attempt to handle such outcomes.

Finally, Table 6 depicts the average run time of the GA per image set; all experiments were conducted on a modern PC. When running on the smaller sets of 432, 540, and 805 pieces, the GA terminated, respectively, after 48.73, 64.06, and 116.18 s, on average. These results are comparable to the running times reported by Pomeranz et al. [16], i.e. 1.2, 1.9, and 5.1 min, respectively, on the same sets. Experimenting with the larger benchmark of 22,755-piece puzzles, the GA terminated, on average, after 13.19 h only. In comparison, Gallagher [9] reported a run time of 23.5 h on a single 9600-pieces puzzle, although he allows also for piece rotation. Despite achieving faster algorithms, the significant increase in run-time with respect to puzzle size has prompted us in various ways to optimize the running time before further experimenting with even larger puzzles.

One objective is to reduce considerably the running time without changing at all the GA itself. As expected, benchmark testing revealed that the most time consuming component of the GA is the creation of a new generation. This includes running the crossover procedure 1000 times on different chromosome pairs to create new offspring. Since the selection of parents and their crossover within the same generation are completely independent, creating a new generation lends itself easily

Table 6 Average run times of both single-threaded and multi-threaded GA per image in every set

No. of Pieces	Run time
<i>Single-threaded</i>	
432	48.73 (sec)
540	64.06 (sec)
805	116.18 (sec)
2360	17.60 (min)
3300	30.24 (min)
5015	61.06 (min)
10,375	3.21 (h)
22,755	13.19 (h)
<i>Multi-threaded</i>	
22,755	3.43 (h)
30,745	6.51 (h)

to concurrent implementation, by utilizing all available CPU cores of the machine. We have implemented a parallelized version of the GA with four threads, each creating 250 offspring. Running on the 22,755- and 30,745-piece puzzle sets, this version terminated after 3.43 and 6.51 h, respectively. In summary, our GA seems to outperform other greedy algorithms on puzzles of smaller and considerably larger size.

4.1 Impact of each crossover phase

Examining more carefully our 3-phase crossover operator raises the interesting question of the specific (relative) impact of each phase. To address this issue, we conducted a series of experiments, using partial crossover variants, each containing one or two of the three phases: “agreed”, “best-buddy”, and “greedy”.

Certain phases may or may not assign a piece, i.e. a piece assignment due to such phases is optional. For example, the “agreed” phase assigns a piece if it is agreed-upon by both parents; if no agreed-upon piece exists, no piece is assigned at all. Other phases, on the other hand, do guarantee piece assignment. Instead of determining whether or not to assign a piece, such phases always assign a piece from the available pool of pieces. For example, the “greedy” phase assigns the most compatible piece available.

Following the previous discussion, it is clear that each variant must consist of at least one phase of a guaranteed assignment (otherwise, if all optional phases do not assign a piece, the algorithm cannot proceed). Since the “greedy” phase always assigns a piece, the implementation of all variants containing this phase is straightforward. For the other variants, we add a pseudo-phase called “random”, which randomly assigns an available piece. Thus, unlike the “greedy only” variant, the “agreed only” variant is actually “agreed + random”. This variant looks at all available boundaries in the growing kernel, and assigns a piece only if both parents agree on it. If no agreed-upon piece exists at any boundary, it randomly selects one

and assigns there an available piece at random (note that this assignment creates a new boundary where an agreed-upon piece might now be placed).

We have experimented with the dataset of Cho et al. [4], which consists of 20 432-pieces puzzles, running the above specified crossover variants of the GA on every image. The results are provided in Table 7.

The poor results obtained by using only the “agreed” phase or the “greedy” phase are not surprising. A GA consisting solely of the “agreed” phase attempts to assemble the puzzle by relying, essentially, on initial generations of random populations, i.e. no significant crossover is expected to take place. The variant consisting only of the “greedy” phase is equivalent to the naive, greedy solution of the problem. On the other hand, the relatively impressive results obtained using only the “best-buddy” phase further underscore the effectiveness of the best-buddies concept, as presented in Pomeranz et al. [16].

The results of additional crossover variants, consisting only of two phases, further ascertain that the “best-buddy” phase seems by far the most critical. All variants containing this component score significantly higher than their counterparts. Most notably, the “best-buddy” phase by itself scores almost 20 % higher than the “no best-buddy” (i.e. “agreed” + “greedy”) variant. Again, this is not surprising, as the best-buddies criterion was shown [16] to be a good indicator for the likelihood of neighboring pieces in the correct puzzle configuration. We assume that a GA based on this criterion utilizes its many chromosomes through the generations to distinguish, eventually, between “genuine” and “false” best-buddy relations.

Having underscored the importance of the “best-buddy” phase, we now examine the two-phase variants containing the “best-buddy” component. The accuracy obtained for both of these variants is over 90 %, on average. It appears that the impact of the “greedy” phase is greater, as it leads to an impressive increase of 2 % when used in conjunction with the “best-buddy” phase. An interesting observation is that for some puzzles the GA achieves actually a better result using a partial operator variant. A more detailed examination reveals that in all of these cases, although the accuracy reported is smaller, the fitness achieved is greater. “Namely, the original operator, which consists of all three phases, might find a global fitness optimum which exceeds that of the correct piece configuration. This is the same kind of anomaly observed with the shifted puzzles in Fig. 5.

The bottom-line accuracy reported by the GA (i.e. the accuracy of the best chromosome in the final generation) is not the sole factor in assessing the importance of a given phase. Another important aspect is the speed of convergence. Our GA always runs for 100 generations, although some puzzles could be completely solved after four or five generations. Moreover, in some puzzles, only slight changes occur in the latter generations, while in others, every generation might be as meaningful as others. Thus, for every run, we record the fitness of the best chromosome in each generation. We report the number of *meaningful generations*, i.e. the number of generations for which the fitness keeps improving. We also compute the average and median fitness improvement through the generations. Let F_i be the fitness of the best chromosome in the i -th significant

Table 7 Accuracy results for 432-piece puzzles, under the neighbor comparison, using our GA with various partial crossover variants

Image No.	Agreed only (%)	Best-buddy only (%)	Greedy only (%)	Best-buddy + greedy (%)	Agreed + greedy (%)	Agreed + best-buddy (%)	Best-GA (%)	Avg-GA (%)	Worst-GA (%)
1	5.23	82.12	31.14	87.96	34.31	76.03	88.44	87.55	86.62
2	5.35	83.33	44.28	84.91	66.06	83.09	85.28	84.73	84.18
3	5.60	100.00	52.19	100.00	76.76	100.00	100.00	100.00	100.00
4	3.53	67.03	32.97	68.49	53.41	66.42	69.46	68.03	65.09
5	4.26	100.00	50.24	100.00	85.89	100.00	100.00	100.00	100.00
6	4.99	89.05	36.86	95.01	67.27	90.63	98.30	97.69	96.72
7	4.74	98.30	44.89	99.64	71.90	99.15	100.00	100.00	100.00
8	4.14	100.00	57.42	100.00	89.66	100.00	100.00	100.00	100.00
9	3.41	99.27	46.72	99.64	80.78	99.15	100.00	99.82	99.64
10	4.62	100.00	45.26	100.00	63.26	100.00	97.81	97.81	97.81
11	4.01	99.64	41.97	99.64	71.65	99.64	97.08	97.08	97.08
12	5.84	97.81	59.49	99.15	85.89	100.00	99.64	99.39	99.03
13	3.65	84.06	38.93	89.66	59.49	84.43	91.12	90.56	90.15
14	5.72	100.00	51.82	100.00	95.01	100.00	99.64	99.64	99.64
15	5.23	87.35	43.67	92.58	63.38	91.61	96.84	96.35	95.86
16	4.01	99.64	43.55	100.00	86.01	99.15	100.00	100.00	100.00
17	6.20	99.64	58.03	99.64	79.81	99.64	99.64	99.64	99.64
18	4.87	89.90	48.30	91.85	72.75	91.00	100.00	95.82	92.82
19	5.11	100.00	55.60	100.00	90.51	100.00	100.00	100.00	100.00
20	5.60	100.00	51.70	100.00	91.73	100.00	100.00	100.00	100.00
Avg	4.81	93.86	46.75	95.41	74.28	94.00	96.16	95.70	95.21

generation; then $\hat{F}_i = F_{i+1} - F_i$ represents the fitness improvement between the two consecutive meaningful generations i and $i + 1$. We report the average and median of $\hat{F}$ per image. The results appear in Table 8. As can be observed, the “greedy” phase greatly improves the speed of convergence.

In conclusion, it appears that all phases play a significant role, assisting both the accuracy achieved and the speed of convergence. In most cases, the original operator’s score is the highest and, on occasion, is even greater than the fitness value of the correct puzzle configuration. These results give rise to other interesting questions, such as the impact of the phases on harder puzzles or the elusive contribution of the agreed phase. We leave these questions for future research.

4.2 Different fitness function

It is well known that a GA relies greatly on the quality of its fitness measure; in the context of the jigsaw puzzle problem, a good fitness measure should be correlative to the piece configuration and allow also for convergence. The previously described anomaly of the shifted puzzles calls for the examination of different fitness functions, as the one used by us proved inexact in some cases. Recent research by Gallagher [9] suggests the *Mahalanobis gradient compatibility* (MGC) as a preferable compatibility measure to those used by Pomeranz et al. [16]. The MGC penalizes changes in intensity gradients, rather than changes in intensity, and learns the covariance of the color channels by using the Mahalanobis distance.

In an attempt to alleviate the shifting phenomenon by exploiting the above properties, we ran the GA on the 20-image set (of 432-piece puzzles) supplied by Cho et al. [4] with the above MGC measure (instead of the dissimilarity used originally) to compute the fitness function. Incidentally, this particular setting enables also a more meaningful comparison with the performance of Gallagher’s algorithm (95.1 % accuracy on the above image set), which uses, of course, the MGC measure. Experimental results are given in Table 9. Employing the GA with the MGC measure improves the average accuracy achieved for this set from 95.1 % (achieved by Gallagher) to 96.03 %. Again, this attests to the superior performance of the GA-based solver.

As noted, unlike other greedy solvers, which use similar compatibility measures, our GA might yield, on occasion, a “better-than-perfect” result. This anomaly could occur, for example, by rearrangement of unicolor surfaces (like the skies) or a misplacement of two or more correctly assembled segments (as illustrated in Fig. 5), due to a high color similarity between their abutting edges (e.g. a sea positioned above the skies). Despite the drawback of the fitness function used, the above phenomenon actually attests to the power of the GA, as it optimizes the fitness function to an extent that has not been observed or reported before. To further explore this phenomenon, we also compared between the original dissimilarity-based GA and a GA variant based on the MGC measure. Shifted puzzle solutions, using the original GA version, were solved correctly. On the other hand, a few images solved correctly with the dissimilarity-based fitness were

Table 8 Convergence speed of the GA with different partial crossover variants; for each image we report the number of meaningful generations and the average and median fitness improvement through the generations

Image No.	Best-buddy only			Best-buddy + greedy			Best-buddy + agreed		
	No. of significant GA generations	Avg. fitness improvement	Med. fitness improvement	No. of significant GA generations	Avg. fitness improvement	Med. fitness improvement	No. of significant GA generations	Avg. fitness improvement	Med. fitness improvement
1	38	3586.15	1651.73	16	9089.37	433.65	32	3490.10	2639.08
2	34	4642.50	2193.71	14	11751.01	13.00	46	3392.00	1582.93
3	29	4775.30	2710.44	6	27306.71	10208.91	29	4906.21	2715.83
4	36	3885.35	1106.94	11	14914.72	2423.29	25	5569.29	3664.47
5	28	5952.24	4742.76	6	32208.79	6950.50	32	5244.04	2059.36
6	38	3488.45	3089.90	16	9067.55	421.01	46	2912.19	1763.02
7	53	3479.68	1826.83	9	22375.16	1773.32	44	4190.63	2705.12
8	31	3439.34	1367.11	5	25793.19	17396.02	33	3184.64	2186.03
9	38	3802.34	2060.44	11	14152.20	72.10	39	3634.47	1185.56
10	38	2777.28	1845.33	6	20229.33	10828.36	39	2619.01	1958.74
11	28	2341.76	1868.33	6	12747.09	5100.99	33	1937.43	1846.94
12	36	3699.86	1840.87	11	12989.78	62.29	39	3370.68	917.29
13	42	2834.61	1406.46	11	12092.63	1372.75	34	3516.87	1966.51
14	33	3436.79	2577.28	5	27851.13	14883.22	29	4012.27	1648.06
15	45	3599.74	2032.63	10	18018.68	2121.83	37	4437.38	2911.77
16	34	3995.88	3334.25	6	26835.21	10911.81	34	4039.69	3742.05
17	33	4325.24	1845.80	8	19592.13	1252.53	25	5850.35	3285.79
18	35	2371.56	1507.59	10	9083.49	514.49	40	2103.28	1260.12
19	22	6590.41	4616.78	5	33977.85	18104.56	23	6211.21	4461.81
20	26	4200.16	1441.01	6	20946.65	4308.45	28	3868.74	2784.35
Avg	34.85	3861.23	2253.31	8.90	19051.13	5457.65	34.35	3924.52	2364.24

Table 9 Accuracy results using our GA with MGC fitness measure vs. dissimilarity fitness

Image No.	GA-MGC	GA-dissimilarity
1	87.10	88.44
2	82.97	85.28
3	100.00	100.00
4	65.69	69.46
5	100.00	100.00
6	100.00	98.30
7	100.00	100.00
8	100.00	100.00
9	100.00	100.00
10	100.00	97.81
11	99.64	97.08
12	99.27	99.64
13	91.85	91.12
14	100.00	99.64
15	94.04	96.84
16	100.00	100.00
17	100.00	99.64
18	100.00	100.00
19	100.00	100.00
20	100.00	100.00
Avg	96.03	96.16

shifted, using the MGC measure. Figure 6 illustrates shifted solutions obtained in various generations using the GA with the MGC measure.

5 Discussion and future work

In this paper we presented a highly effective jigsaw puzzle solver, based for the first time on evolutionary computation. The automatic solver developed is capable of reconstructing puzzles of up to 30,745 pieces (i.e. more than twice the puzzle size that has been attempted before). The performance observed is comparable, if not superior to other known solvers, in most of the cases examined. Also, we created new sets of very large images for benchmark testing of this and other solvers, and supplied both the image sets and our results for the benefit of the community [18].

By introducing a novel crossover technique, we were able to arrive at a most effective solver. Our approach could prove useful in further utilization of GAs (e.g. [20, 21]) for solving more difficult variations of the jigsaw problem (including unknown piece orientation, missing and excessive puzzle pieces, unknown puzzle dimensions, and three-dimensional puzzles), and could also assist in the design of GAs in other problem domains.

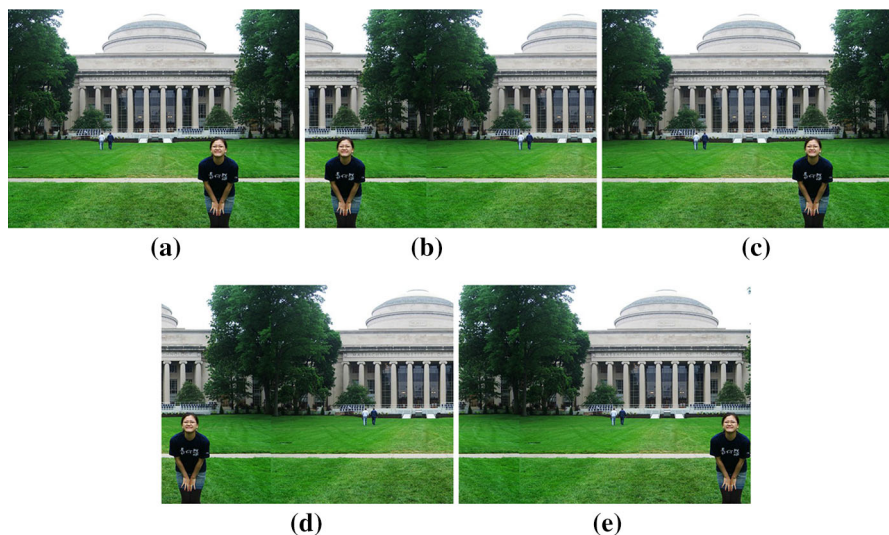


Fig. 6 Shifted puzzle solution obtained using the GA with MGC fitness function; best chromosome from selected generations is shown, each with a higher fitness than preceding chromosomes. **a** Original image. **b** Generation 6. **c** Generation 9. **d** Generation 10. **e** Generation 16 (final)

References

1. T. Altman, Solving the jigsaw puzzle problem in linear time. *Appl. Artif. Intell. Int. J.* **3**(4), 453–462 (1989)
2. B. Brown, C. Toler-Franklin, D. Nehab, M. Burns, D. Dobkin, A. Vlachopoulos, C. Doumas, S. Rusinkiewicz, T. Weyrich, A system for high-volume acquisition and matching of fresco fragments: Reassembling Thera wall paintings. *ACM Trans. Graph.* **27**(3), 84 (2008)
3. S. Cao, H. Liu, S. Yan, Automated assembly of shredded pieces from multiple photos, in *IEEE International Conference on Multimedia and Expo* (2010), pp. 358–363
4. T. Cho, S. Avidan, W. Freeman, A probabilistic image jigsaw puzzle solver, in *IEEE Conference on Computer Vision and Pattern Recognition* (2010), pp. 183–190
5. T. Cho, M. Butman, S. Avidan, W. Freeman, The patch transform and its applications to image editing, in *IEEE Conference on Computer Vision and Pattern Recognition* (2008), pp. 1–8
6. A. Deever, A. Gallagher, Semi-automatic assembly of real cross-cut shredded documents, in *IEEE International Conference on Image Processing* (2012), pp. 233–236
7. E. Demaine, M. Demaine, Jigsaw puzzles, edge matching, and polyomino packing: connections and complexity. *Graphs Comb.* **23**, 195–208 (2007)
8. H. Freeman, L. Garder, Apictorial jigsaw puzzles: the computer solution of a problem in pattern recognition. *IEEE Trans. Electron. Comput.* **EC-13**(2), 118–127 (1964)
9. A. Gallagher, Jigsaw puzzles with pieces of unknown orientation, in *IEEE Conference on Computer Vision and Pattern Recognition* (2012), pp. 382–389
10. D. Goldberg, C. Malon, M. Bern, A global approach to automatic solution of jigsaw puzzles. *Comput. Geom. Theory Appl.* **28**(2–3), 165–174 (2004)
11. E. Justino, L. Oliveira, C. Freitas, Reconstructing shredded documents through feature matching. *Forensic Sci. Int.* **160**(2), 140–147 (2006)
12. D. Koller, M. Levoy, Computer-aided reconstruction and new matches in the forma urbis romae. *Bullettino della Commissione Archeologica Comunale di Roma. Supplementi.* **15**, 103–125 (2006)
13. W. Marande, G. Burger, Mitochondrial DNA as a genomic jigsaw puzzle. *Science* **318**(5849), 415–415 (2007)

14. M. Marques, C. Freitas, Reconstructing strip-shredded documents using color as feature matching, in *ACM Symposium on Applied Computing* (2009), pp. 893–894
15. A.Q. Morton, M. Levison, The computer in literary studies, in *IFIP Congress* (1968), pp. 1072–1081
16. D. Pomeranz, M. Shemesh, O. Ben-Shahar, A fully automated greedy square jigsaw puzzle solver, in *IEEE Conference on Computer Vision and Pattern Recognition* (2011), pp. 9–16
17. D. Pomeranz, M. Shemesh, O. Ben-Shahar, A fully automated greedy square jigsaw puzzle solver MATLAB code and images (2011). <https://sites.google.com/site/greedyjigsawsolver/home>
18. D. Sholomon, O.E. David, N.S. Netanyahu, Datasets of larger images and GA-based solver's results on these and other sets. <http://www.cs.biu.ac.il/~nathan/Jigsaw>
19. D. Sholomon, O.E. David, N.S. Netanyahu, A genetic algorithm-based solver for very large jigsaw puzzles, in *IEEE Conference on Computer Vision and Pattern Recognition* (2013), pp. 1767–1774
20. D. Sholomon, O.E. David, N.S. Netanyahu, Genetic algorithm-based solver for very large multiple jigsaw puzzles of unknown dimensions and piece orientation, in *ACM Conference on Genetic and Evolutionary Computation* (2014), pp. 1191–1198
21. D. Sholomon, O.E. David, N.S. Netanyahu, A generalized genetic algorithm-based solver for very large jigsaw puzzles of complex types, in *AAAI Conference on Artificial Intelligence* (2014), pp. 2839–2845
22. K. Son, J. Hays, D.B. Cooper, Solving square jigsaw puzzles with loop constraints, in *European Conference on Computer Vision 2014* (Springer, 2014), LNCS 8694, pp. 32–46
23. F. Toyama, Y. Fujiki, K. Shoji, J. Miyamichi, Assembly of puzzles using a genetic algorithm, in *International Conference on Pattern Recognition* (2002) (Vol. 4), pp. 389–392
24. C.S.E. Wang, Determining molecular conformation from distance or density data, Ph.D. thesis, Massachusetts Institute of Technology, Department of Electrical Engineering and Computer Science (2000). <http://theses.mit.edu/Dienst/UI/2.0/Describe/0018.mit.theses/2000-2>. Also available online at the MIT theses online homepage <http://thesis.mit.edu/>
25. X. Yang, N. Adluru, L.J. Latecki, Particle filter with state permutations for solving image jigsaw puzzles, in *IEEE Conference on Computer Vision and Pattern Recognition* (2011), pp. 2873–2880
26. Y. Zhao, M. Su, Z. Chou, J. Lee, A puzzle solver and its application in speech descrambling, in *WSEAS International Conference Computer Engineering and Applications* (2007), pp. 171–176